

Chapter 1 Experimental Results

This study conducts experiments in an OpenAirInterface (OAI) nFAPI end-to-end environment to verify the proposed delay management method. To ensure the statistical reliability of the evaluation, each throughput value reported represents the average of 180 data points (3 independent trials $\times$ 60 samples per trial).

To validate the hypothesis and the effectiveness of the proposed dynamic trigger scheduling algorithm, the experimental scenarios are arranged as follows: Section 1.1 outlines the hardware and software environment as well as the automated evaluation framework. Section 1.2 validates the Exponentially Weighted Moving Average (EWMA) mechanism for the PNF arrival offset (Δt_{arrive}). Section 1.3 benchmarks the performance of peak throughput and MAC layer Round-Trip Time (RTT) against static baselines. Section 1.4 evaluates the stability of the proposed method under varying degrees of network congestion.

1.1 Experimental Scenario

The testbed environment connects a series of high-performance servers, radio units, and commercial smartphones. The detailed hardware and software configuration of the end-to-end network deployment is shown in the table below.

To ensure consistent data quality for automated measurement and analysis, this study developed an automated evaluation rApp running on the

Table 1.1: Testbed Node Configuration

Unit	Specification
Core Network (CN)	Open5GS [?]
Software Protocol Stack / Branch	OpenAirInterface [?] (2026.w13)
VNF Server	Intel® Xeon® Gold 6226R [?] with NetXtreme BCM5719 NIC [?,?]
PNF Server	Intel® Xeon® Gold 6433N [?] with Intel I210 NIC [?,?]
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
Commercial Off-The-Shelf (COTS) UE	Samsung Galaxy S20 [?]

Table 1.2: Wireless System Parameters

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul Interface	O-RAN F release

Non-Real-Time RAN Intelligent Controller (Non-RT RIC) within the O-RAN Service Management and Orchestration (SMO) framework [?,?], as shown in Figure 1.1. This rApp automatically starts the VNF and PNF. Once the gNB is successfully established, the rApp automatically connects to the control PC via SSH, toggles the UE’s airplane mode using the Android Debug Bridge (ADB) [?], and then launches an iPerf [?] server on the Core Network server via SSH. Subsequently, it automatically executes a series of predefined iPerf test cases. After that, the rApp collects all log files via O1 SFTP and uses Python scripts to automatically plot charts for subsequent data analysis. During the experiments, we also used tcpdump [?] for packet sniffing, and utilized Linux PTP [?] and TSN time synchronization mechanisms [?] to ensure baseline timing among network nodes, while underlying data transmission relied on DPDK [?] for packet acceleration. A key capability of this framework is the ability to automate network delay simulations on the MAC feedback link ($T_5 - T_4$) and fronthaul link ($T_2 - T_1$) using Linux Traffic Control (TC), which is crucial for verifying the stability of the disaggregated architecture under non-deterministic latency.

1.1.1 Software Implementation and CI/CD Validation

To ensure that the proposed adaptive timing control and nFAPI modifications work well with current O-RAN standards, we have submitted our code to the official OpenAirInterface (OAI) [?] repository. In addition to our local end-to-end evaluation using the rApp testing framework, the proposed software implementation was also pushed as a merge request (MR) to the official OAI repository. The code successfully passed vari-

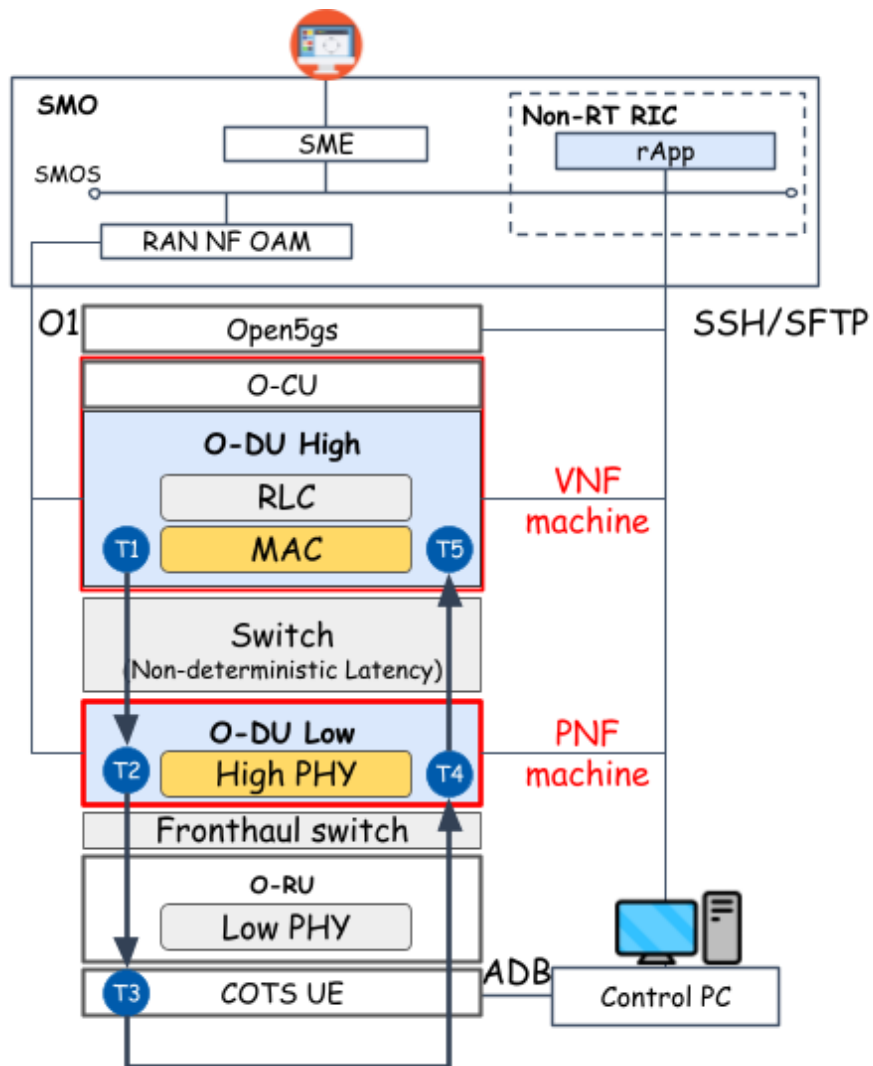


Figure 1.1: Architecture of the rApp automated evaluation framework.

ous automated testing procedures in the OAI Continuous Integration and Continuous Deployment (CI/CD) pipeline [?].

In the software build and deployment stage (image building processes), the proposed implementation shows high compatibility across different hardware platforms. As shown in Table 1.3, the image building and cross-compilation tests have all passed. These tests include x86-based Ubuntu and Red Hat Enterprise Linux (RHEL) [?] systems, as well as ARM64-based NVIDIA Jetson edge platforms. These results show that the proposed method is mature enough to be deployed on different commercial hardware devices.

Table 1.3: OAI CI/CD Cross-Platform Compilation and Image Building Results

Stage	Job / Builder	Target and Hardware Description	Result
x86 Build	Ubuntu-Image-Builder	Build OAI network function images on standard x86 servers	Passed
ARM Build	Ubuntu-ARM-Image-Builder	Build images on ARM64 for ARM-based edge servers	Passed
Edge AI Build	Ubuntu-Jetson-Image-Builder	Build light-weight images for NVIDIA Jetson platforms	Passed
RHEL Build	RHEL-Cluster-Image-Builder	Build images for Red Hat Enterprise Linux (RHEL) clusters	Passed
Cross-Compile	ARM-Cross-Compile	Verify cross-compilation from x86 to ARM64 architecture	Passed

After cross-platform builds, the code was tested in OAI system integration and physical layer dynamic tests. Although OAI includes many tests for special scenarios (such as Non-Terrestrial Networks, NTN), our

method has passed several key tests in the main 5G branches. Table 1.4 summarizes the key integration and simulation tests that our code passed.

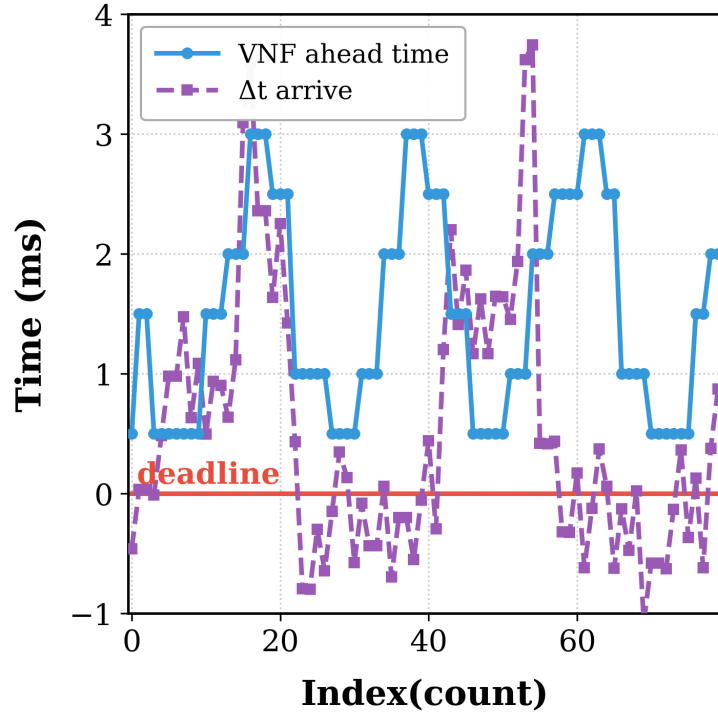
Table 1.4: Summary of System Integration and Physical Layer Validation

Validation Scope	Passed Job Name	Description and Highlights
E2E 5G SA and RIC Integration	OAI-FLEXRIC-RAN-Integration	Run in 5G SA mode, integrate with 5G Core and FlexRIC (near-RT RIC) [?] with low packet loss. This validation uses the E2 interface protocol [?].
Commercial UE and RF Integration	RAN-NSA-B200-Module-LTEBOX RAN-SA-B200-Module-SABOX	Run with USRP B200 and commercial Quectel modules to meet strict timing in real RF environments.
Third-Party RF Frontend Compatibility	RAN-SA-AW2S-CN5G	Run end-to-end tests with third-party AW2S RF equipment (20MHz TDD SA) [?] and Amarisoft UE.
Handover Stability	RAN-SA-Handover-CN5G	Pass F1-interface and N2-interface handover tests [?] under 5G SA to keep control plane stable.
5G L1 and Channel Compatibility	RAN-Channel-Simulation RAN-PhySim-GraceHopper-5G	Pass physical layer simulations (including LDPC, Polar, PBCH, MIMO, MCS) to prove timing changes do not break 3GPP rules [?]. Testing was also performed on GPU-accelerated platforms [?].

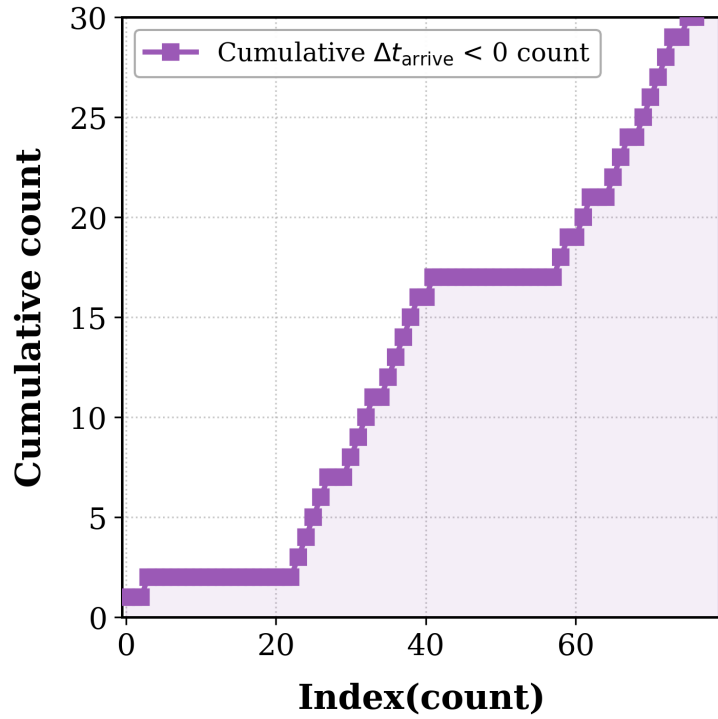
The results of the builds and key tests show that the proposed delay management mechanism is not only correct in theory, but also highly compatible in practice. It can serve as a stable open-source baseline for future optimizations in the O-RAN nFAPI community.

1.2 Scenario I: EWMA Validation for PNF Arrival Offset (Δt_{arrive})

This scenario verifies the effectiveness of applying the Exponentially Weighted Moving Average (EWMA) to smooth the PNF arrival offset (Δt_{arrive}). Due to network jitter and processing delays, the raw timing offset received from the PNF can exhibit high variability. Figure 1.2 illustrates the impact of EWMA processing. Without EWMA (Figure 1.2a), the arrival offset fluctuates drastically, which can lead to unstable scheduling decisions. By applying EWMA (Figure 1.2b), high-frequency jitter is effectively filtered, providing a stable baseline for the timing adjustment algorithm. Furthermore, as shown in Figures 1.2c and 1.2d, the smoothed data allows the system to make relatively stable judgments. When Δt_{arrive} begins to show a downward trend, the algorithm steadily identifies this trend and actively advances more time (S_{ahead}) to maintain synchronization stability.

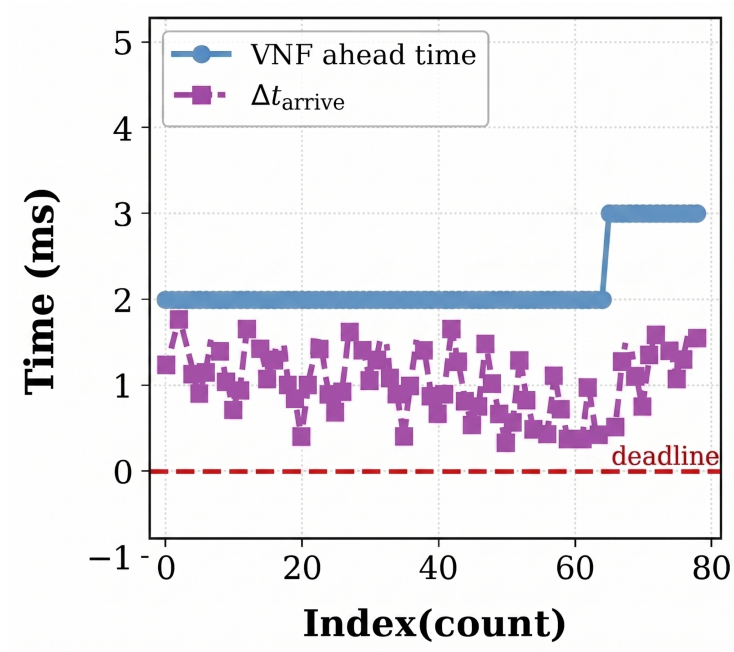


(a) Without EWMA

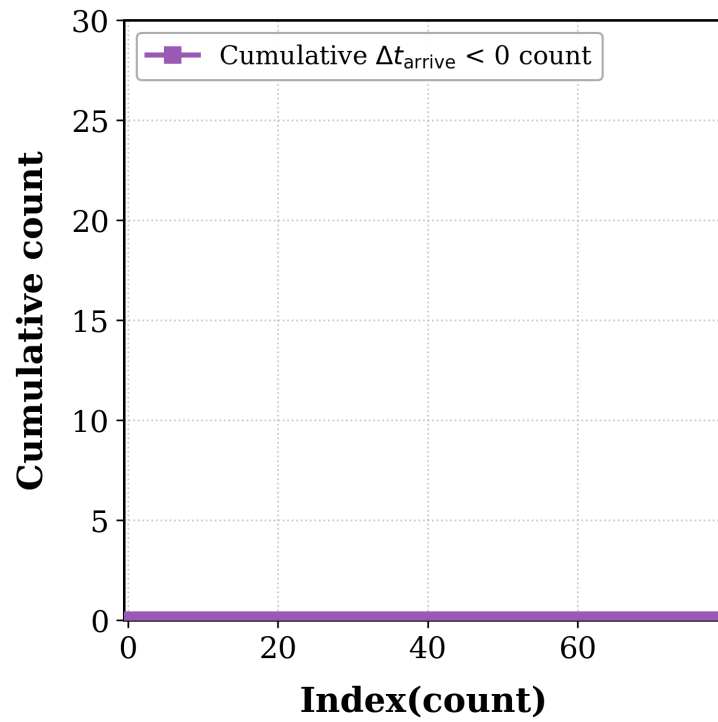


(b) With EWMA

Figure 1.2: EWMA processing validation and active adjustment for PNF arrival offset (Δt_{arrive}).



(c) Trend Observation



(d) Active Advance Adjustment

Figure 1.2: EWMA processing validation and active adjustment for PNF arrival offset (Δt_{arrive}) (cont.).

1.3 Scenario II: Performance Benchmarking

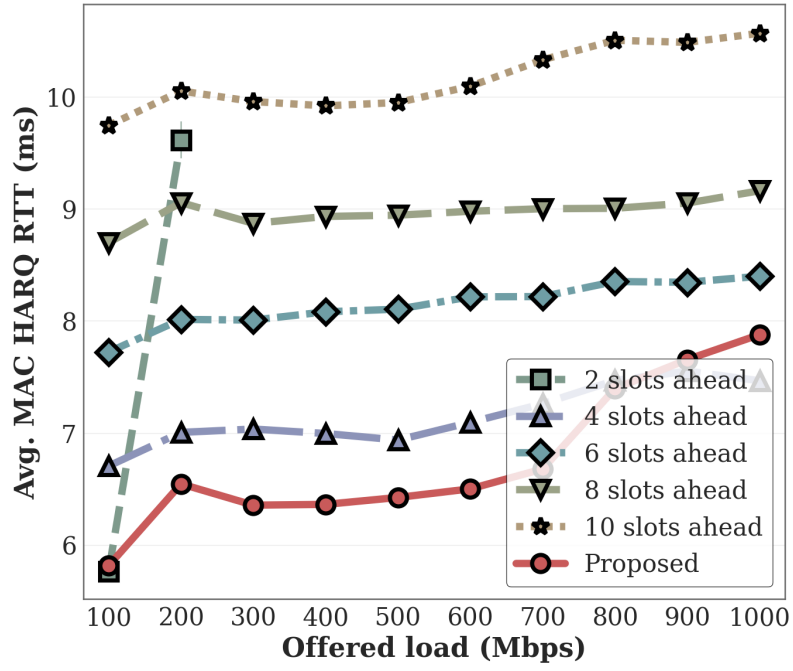
In this scenario, we evaluate the benchmark performance of the proposed adaptive timing mechanism against a standard static slots ahead configuration. Figure 1.3 presents the performance evaluation in terms of MAC layer latency and system throughput.

Figure 1.3 displays the performance evaluation under varying Offered Load. The proposed algorithm prioritizes ensuring optimal throughput, consistently maintaining optimal throughput performance under all load conditions. Simultaneously, while ensuring this optimal throughput, the algorithm maximizes the selection of the optimal slots ahead to optimize the MAC layer HARQ Round-Trip Time (RTT), effectively avoiding unnecessary extra overhead.

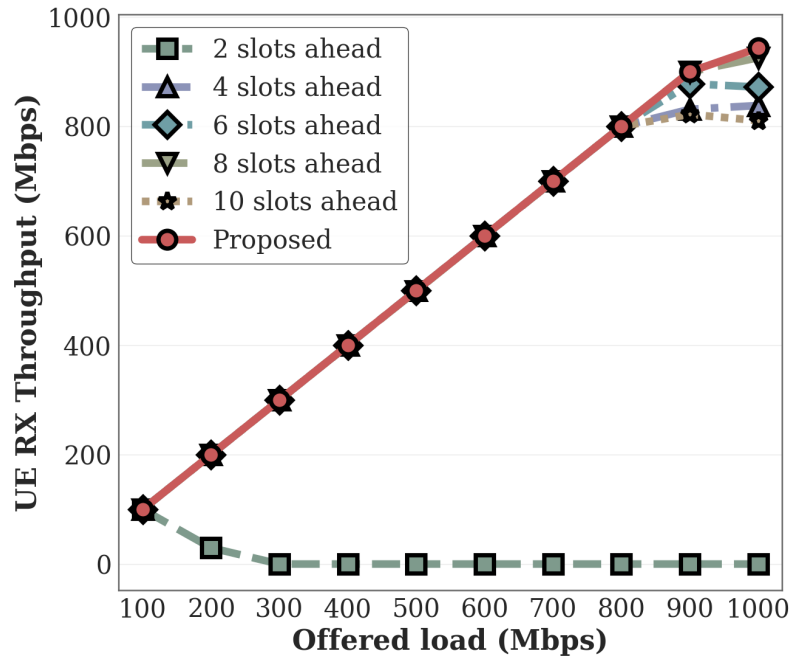


1.4 Scenario III: Stability under Network Congestion

To test the system's stability, we utilized Linux TC to inject artificial network congestion into the $T_2 - T_1$ and $T_5 - T_4$ segments, altering the One-Way Delay (OWD) and jitter. As shown in Figure 1.4a, when the delay exceeds the fixed timing window, the static configuration experiences a severe drop in throughput and frequent connection resets. In contrast, the proposed adaptive timing control dynamically expands the slots ahead (S_{ahead}). This expansion allows the MAC scheduler to trigger messages in advance, ensuring that messages can still arrive at the PNF within the valid



(a) Throughput Optimization

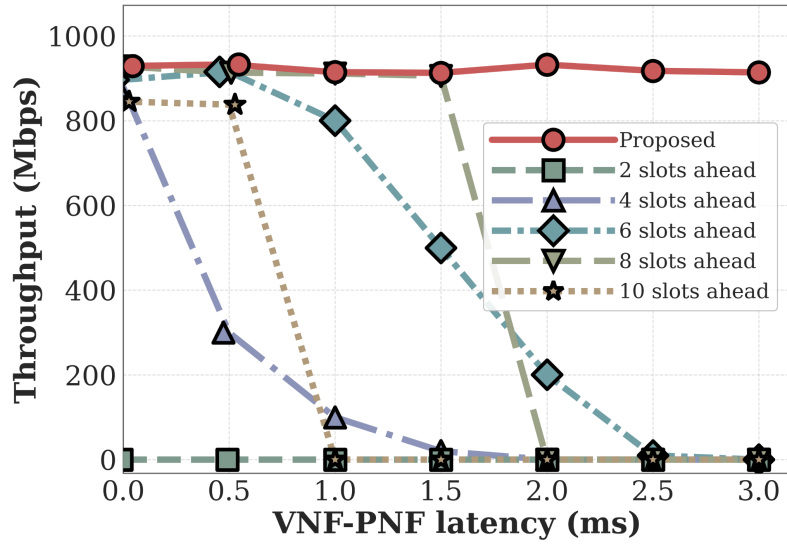


(b) MAC Latency and Reliability

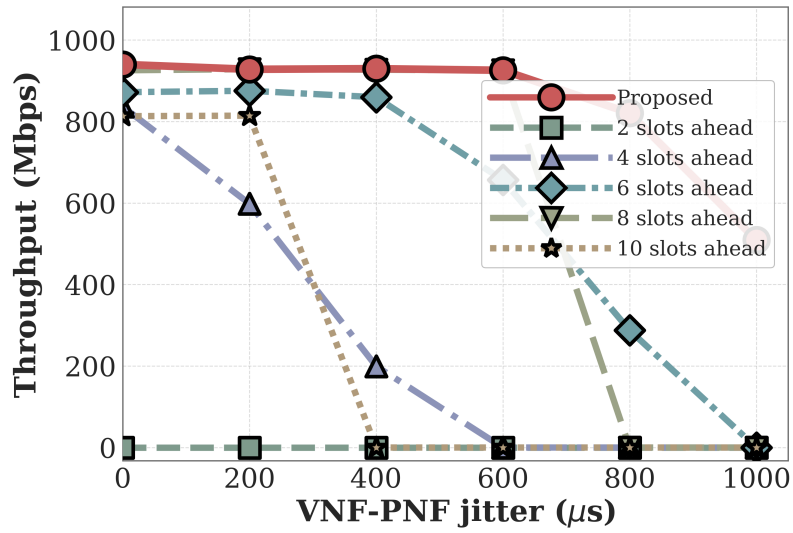
Figure 1.3: Performance Evaluation: MAC RTT Efficiency and System Throughput.

timing window despite the increased transmission delay. Therefore, even at congestion levels that would previously cause complete synchronization failure, the system can maintain stable throughput.

To further evaluate the impact of unpredictable network jitter between the VNF and PNF, we perform a second experiment using Linux TC to simulate random jitter on the fronthaul link. In this test, we run iPerf UDP traffic with an offered load of 1 Gbps to fully saturate the downlink transmission. We measure the resulting downlink (DL) receiver (RX) throughput at the UE, comparing different fixed slot-ahead configurations against our proposed adaptive timing mechanism. As shown in Figure 1.4b, when network jitter is high and unpredictable, fixed slot-ahead settings cannot handle the timing variations, leading to packet losses, sync failures, and a major drop in throughput. On the other hand, the proposed adaptive method dynamically adjusts the scheduling advance, keeping the transmission timing aligned. As a result, the UE achieves and maintains stable and high DL RX throughput even under severe jitter conditions.



(a) System Stability under Network Congestion



(b) Throughput Performance under Unpredictable Jitter

Figure 1.4: System stability and throughput performance under network congestion and jitter.